

Coding & Solution

Date	4 July 2026
Team ID	
Project Name	A Comprehensive Measure of Well-Being
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/mudeganeshnaik/AI-ML--Human-Development-Index
Programming Language(s)	Python
Framework(s) Used	Flask
Key Features Implemented	Data cleaning, ML-based HDI prediction model, User-friendly Web Interface (HTML/Bootstrap), Real-time prediction analysis.
Pending / Incomplete Features	None (All core functional requirements are met).
Setup / Run Instructions	1.Prerequisites: Ensure Python (3.x) is installed on your system 2.Clone Repository: Download the project files from the GitHub link provided above. 3.Install Dependencies: Navigate to the project folder and run the following command in your terminal: -pip install -r requirements.txt 4.Run Application: Execute the main application script: - python app.py 5.Access Web UI: Open your browser and go to http://127.0.0.1:5000/ to view the dashboard and interact with the HDI prediction tool.

Code Quality Checklist

S.N o	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The codebase follows a clean, modular structure using Flask for the backend, which ensures high maintainability and scalability. All core ML logic and data processing scripts are properly documented with inline comments for clarity. Exception handling (try-except blocks) has been implemented to ensure the application remains stable during unexpected input or runtime errors. The project is fully versioned using Git/GitHub to facilitate collaborative development and tracking.

app.py : the source code

```
# =====
# IMPORTING THE REQUIRED LIBRARIES# =====
import os
import pickle
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from flask import Flask, render_template, request
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Hiding warnings to match clean output format
warnings.filterwarnings("ignore", category=UserWarning)

# =====
# DATA LOADING & MACHINE LEARNING MODEL SETUP
# =====
full_dataset = {
    'HDI Rank': list(range(1, 61)),
    'Country': [
        'Norway', 'Australia', 'Switzerland', 'Germany', 'Denmark', 'Singapore', 'Netherlands', 'Ireland', 'Iceland',
        'Canada', 'United States', 'Hong Kong', 'New Zealand', 'Sweden', 'Liechtenstein', 'United Kingdom', 'Japan',
        'Korea', 'Israel', 'Luxembourg', 'France', 'Belgium', 'Finland', 'Austria', 'Slovenia', 'Italy', 'Spain',
        'Czech Republic', 'Greece', 'Brunei', 'Estonia', 'Andorra', 'Cyprus', 'Malta', 'Qatar', 'Lithuania', 'Poland',
        'Saudi Arabia', 'Slovakia', 'Portugal', 'Latvia', 'Hungary', 'Bahrain', 'Chile', 'Croatia', 'Argentina',
        'Oman', 'Russia', 'Montenegro', 'Romania', 'Kuwait', 'Belarus', 'Bahamas', 'Kazakhstan', 'Uruguay', 'Libya',
        'Malaysia', 'Barbados', 'Trinidad and Tobago', 'Seychelles'
    ]
}
```

```

81.6, 82.4, 83.0, 80.9, 80.2, 82.6, 81.6, 80.7, 82.3, 81.5, 78.9, 83.7, 81.4, 82.0, 79.2, 80.5, 83.5, 80.4,
81.8, 81.1, 82.2, 80.8, 80.8, 81.4, 80.5, 82.8, 82.8, 78.6, 81.0, 78.8, 77.0, 81.1, 80.3, 81.5, 78.2, 74.6,
77.4, 74.3, 76.3, 80.9, 74.2, 75.2, 76.7, 81.7, 77.3, 76.3, 76.6, 70.1, 76.2, 74.8, 74.3, 71.3, 75.4, 69.4,
77.0, 71.6, 74.7, 75.3, 70.1, 73.1
],
'Mean years of schooling': [
13.4, 13.2, 13.4, 13.1, 12.7, 12.9, 11.9, 12.2, 12.0, 12.3, 11.6, 12.5, 12.8, 12.1, 13.0, 12.2, 12.4, 12.1,
12.6, 11.8, 11.5, 11.4, 10.3, 11.3, 12.1, 10.9, 9.8, 12.3, 10.5, 9.1, 12.5, 10.4, 11.6, 10.3, 9.8, 12.7, 11.9,
8.7, 12.2, 8.9, 11.7, 11.6, 9.4, 9.8, 11.0, 9.9, 8.1, 12.0, 11.3, 10.8, 7.3, 12.0, 10.9, 11.4, 8.6, 7.3, 10.1,
9.4, 10.9, 9.4
],
'Gross national income (GNI) per capita': [
66378, 42822, 59161, 44033, 41188, 76628, 45892, 43764, 38165, 41484, 52947, 51048, 33261, 41198, 79292, 39154,
36927, 32185, 33621, 56324, 38054, 41243, 38868, 43609, 27903, 33573, 32045, 28144, 24502, 72570, 26362, 43260,
29459, 29500, 129916, 26006, 24117, 52821, 26764, 25757, 22589, 21848, 37236, 21290, 20291, 22054, 34402, 22352,
15410, 19428, 83971, 16423, 21336, 20876, 19148, 14911, 22762, 15336, 25619, 13702
],
'Internet users': [
96.8, 93.0, 86.8, 88.0, 96.0, 82.1, 93.3, 81.1, 98.2, 87.1, 87.4, 73.0, 89.4, 91.0, 96.1, 91.6, 91.1, 89.9,
79.8, 93.1, 83.6, 85.0, 92.4, 81.0, 72.8, 62.0, 76.2, 79.7, 63.2, 71.2, 88.4, 96.9, 71.7, 73.1, 91.5, 71.4,
67.5, 64.7, 79.9, 68.4, 79.0, 72.8, 93.5, 64.3, 68.6, 69.4, 71.1, 73.4, 64.6, 54.1, 78.7, 62.2, 78.0, 70.4,
64.6, 21.1, 67.5, 76.1, 65.1, 51.0
],
'HDI': [
0.94, 0.93, 0.93, 0.93, 0.92, 0.92, 0.92, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.91, 0.90,
0.90, 0.90, 0.89, 0.89, 0.89, 0.89, 0.89, 0.89, 0.88, 0.87, 0.86, 0.86, 0.86, 0.85, 0.85, 0.85, 0.85, 0.84,
0.84, 0.84, 0.84, 0.83, 0.83, 0.82, 0.82, 0.82, 0.82, 0.82, 0.80, 0.80, 0.80, 0.80, 0.79, 0.79, 0.79,
0.79, 0.78, 0.78, 0.78, 0.78, 0.77
]
]
Development = pd.DataFrame(full_dataset)

# Clean and prepare data
X_numeric = Development[['HDI Rank', 'Life expectancy', 'Mean years of schooling', 'Gross national income (GNI) per capita', 'Internet users']].copy()
X_numeric = X_numeric.fillna(X_numeric.mean())
y = Development['HDI']

# Training the model
X_train, X_test, y_train, y_test = train_test_split(X_numeric, y, test_size=0.35, random_state=42)
reg = LinearRegression()
reg.fit(X_train, y_train)

# Save the model
pickle.dump(reg, open('HDI.pkl', 'wb'))

# =====
# FLASK APP ROUTING
# =====
app = Flask(__name__)
model = pickle.load(open('HDI.pkl', 'rb'))

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/Prediction')
def prediction():
    return render_template('indexnew.html')

@app.route('/predict', methods=['POST'])
def predict():

```

```

try:
    # Get data from form
    rank = float(request.form.get('HDI Rank', 1.0))
    life_exp = float(request.form.get('Life expectancy', 75.0))
    schooling = float(request.form.get('Mean years of schooling', 10.0))
    gni = float(request.form.get('Gross national income (GNI) per capita', 20000.0))
    internet = float(request.form.get('Internet users', 60.0))

    features = [np.array([rank, life_exp, schooling, gni, internet])]
    prediction_output = model.predict(features)
    output = round(prediction_output[0], 3)

    return render_template('indexnew.html', prediction_text=f'Predicted HDI is: {output}')

except Exception as e:
    return render_template('indexnew.html', prediction_text='Error in prediction, please check your input values.')

if __name__ == '__main__':
    app.run(debug=True, use_reloader=False, port=5000)

```